VFS eVTOL Drone (“Insert Drone Name”) Critical Design Review

Adarsh Sheth, Zixiao Zhang, Aaron Esquivel, Juan Polanco

Mission Requirements

VFS IDC: Mission Requirements

- **The Mission:** A high altitude search and rescue objective on Mt. Rainier requires the locating a lost hiker and providing them with a resupply. An eVTOL drone must be designed that can traverse 14 miles to the search zone, perform a search and locate the hiker within 30 minutes in dense fog (½ standard detection), and drop the 5 lb payload in high winds (20-40 knots) within a margin of error of 15 feet. The drone must operate at 10,000 feet MSL and complete its mission in 50 minutes or less.

- **Holistic Design Statement:** The success of the mission entirely depends on the survivability of the drone. An efficient drone's design depends on its ability to fly in a stable manner in high winds and to be able to sustain a 30 minute search and rescue at 10,000 feet air density without depleting significant power.

Requirement 1: Stability

- Stability was chosen because the mountainous conditions in the forested area requires high control authority while maintaining aerodynamic traits that mitigate the effects of wind gusts in low atmospheric pressure. All of this prevents terrain collision and ensures the ease of locating the hiker.

- We're assigning stability a weight of 30 out of 100 because it is of the utmost importance to ensure that the drone maintains a predictable flight in 40-knot gusts and allows the flight computer to accurately calculate the payload trajectory for a precise delivery.

Requirement 2: Hover Power

- Hover Power was chosen for its determination on the configuration of the drone that would deplete the least amount of power while providing the most about of thrust in a 10,000 ft MSL 50 minute flight to where the drone can work to stabilize itself. This requirement is necessary to ensure the drone can generate adequate thrust without depleting the battery before the search is complete.

- We're assigning Hover Power with a weight of 25 out of 100 because this factor acts as a determinate as to whether or not a drone can support a 5lb payload by identifying its power consumption rate to its body proportionality to ensure it can remain airborne for its allotted search time.

Requirement 3: Endurance

- Endurance was chosen because the drone has to be able to both sustain the conditions of its environment while also having a minimal change to battery consumption. It is relatively important to be able to identify an estimated battery life based on the conditions of the drone in order to sustain a 14-mile transit and 30 minute search.

- We're assigning Endurance to have a weight of 10 out of 100 because this requirement acts as the acts as the minimum threshold to what type of batteries and sensors can be equipped. Since the Hover Power requirement already addresses the highest rate of energy drain, a more rigorous approach to endurance ensures that the chosen configuration maintains significant energy to perform its trip, sensing, and payload drop.

Requirement 4: Stall Speed

- Stall Speed was chosen to define the minimum safe airspeed for wing-borne flight and hovering a minimum of 100 ft AGL. An optimal pattern search in the dense fog requires a sufficient speed to intake data and locate the hiker.

- We're assigning Stall Speed to have a weight of 10 out of 100 because it acts as a sensor optimization threshold, where we can identify the lowest speed we can cruise and hover in order to visually identify the hiker through dense fog and timber. An optimal configuration ought to have a lower stall speed.

Requirement 5: Mechanical Complexity/Manufacturability

- Mechanical Complexity was chosen over the feasibility of actually designing and working with the configuration. This is meant to address the risk of component failure in harsh time-sensitive environments. The requirement prioritizes designs with fewer moving parts to reduce the likelihood of jamming.

- We're assigning Mechanical Complexity with a weight of 10 out of 100 because it acts as the reliability factor. While high complexity can offer efficiency gains, it introduces significantly more points of critical failure; therefore, this weight is chosen so a configuration is weighed on whether or not it can actually complete the task of saving a life.

Requirement 6: Payload Delivery Accuracy

- Payload Delivery Accuracy was chosen to evaluate the collective ability of the configuration to hover, change altitude in minimal space, and the stability in high crosswinds in order to drop the payload within the 15 ft margin of error.

- We're assigning Payload Delivery Accuracy a weight of 10 out of 100 because it checks the efficiency of the mission. This requirement ensures that all components of the drone can collectively work together to create an optimal release mechanism versus a simple free-fall.

Requirement 7: Cruise Speed

- Cruise Speed was chosen for the approximately 20 minute window the drone has to travel to the search site. This ensures that the drone has the capability to reach its destination in adequate time and perform its search.

- We're assigning Cruise Speed with a weight of 5 out of 100 because of the low priority for a significant amount of speed. On average, the drone ought to travel a minimum of approximately 45 miles per hour, making any higher speed overkill and withholding of other parameters.

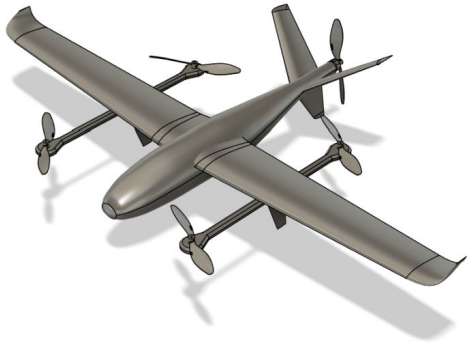
Design Matrix

VFS IDC: Final Four Design Matrix

Parameter	Weight	Lift + Cruise ✓	Tiltrotors ✗	Quad Bi-Plane ✗	Tiltwing ✗
Stability (Control Authority and Dynamic Response)	30	3	2	4	1
Hover Power	25	4	2	3	1
Endurance (Flight Time)	10	4	1	2	3
Stall Speed	10	3	1	4	2
Mechanical Complexity / Manufacturability	10	4	3	2	1
Payload Delivery Accuracy	10	4	3	2	1
Cruise Speed	5	2	4	1	3
Total	100	350	210	300	140

VFS IDC: Lift + Cruise Breakdown

- **Stability (3)**: The configuration minimizes crosswind vulnerability by maintaining a horizontal profile in hover. This effectively reduces the lateral drag, minimizing the "Sail Effect" and optimizing symmetric control authority for gust rejection [1], [7].
- **Hover Power (4):** The configuration utilizes large diameter rotors to reduce the disk loading. This creates strong hover power whilst also increasing the battery efficiency during the 30 minute search phase when compared to high disk loading mechanisms [2].
- **Endurance (4):** Although the stationary lift rotors during cruise create parasitic drag, the heavy energy saving during hover balance out overall energy consumption, resulting in an effective mission. By isolating the vertical lift system, we can minimize power consumption when compared to the tilt wing and tilt rotor mechanisms [3].
- **Stall Speed (3):** The independent lift system itself provides an active safety net against stalling which allows the drone to remain stable at lower speeds overall optimizing sensor performance [1].
- **Mechanical Complexity (4):** The design itself decreases the reliance on moving parts which simplifies the manufacturing process and overall cost of production.
- **Payload Delivery Accuracy (4):** The configuration allows for a separation between altitude control and position control which allows the flight controller to better counteract the wind without pitching the payload. This stability overall minimizes the chance of errors on release of the payload [1].
- **Cruise Speed (2):** The configuration scores on the lower end for cruise speed as a result of the parasitic drag resulting from the stationary vertical rotors during the cruise phase. This drag overall reduces cruise efficiency as a trade off for stability [1], [2].



VFS IDC: Tiltrotor Breakdown

- **Stability (2)**: The configuration is more susceptible to crosswinds as it is often less aerodynamic and focuses on the power of the rotors. This can create a “Barn Door” effect in which the overall gust rejection is minimal when compared to the other configurations [4].
- **Hover Power (2):** Tiltrotors often require higher disk loading in order to maintain an efficient cruise propellers sizing, this results in high power but also a significant power consumption [3].
- **Endurance (1):** In a long range cruise, this configuration is decently aerodynamic, but it endures a heavy energy penalty during hover which outweighs energy efficiency [3].
- **Stall Speed (1):** The transition phase between hovering and cruise is typically high risk as a result of the needed conditions in the “Conversion Corridor.” Overall, this makes the stall speed much higher, which is unideal for our drone [4].
- **Mechanical Complexity (3):** The design would require high torque tilting mechanisms that must operate flawlessly in humid conditions which overall adds to the complexity of the design. This would introduce extra critical failure points in addition to the stall speed [4].
- **Payload Delivery Accuracy (3):** While more prone to tilting in hover, the configuration has to utilize active thrust vectoring to reject wind gusts. By tilting the rotors rather than tilting the aircraft, a lateral force can be generated to keep a stable trajectory for the payload ejection [4], [6].
- **Cruise Speed (4):** This configuration achieves high cruise speeds and range due to its lower-drag profile and higher rotor thrust, but effectively loses more efficiency at slower speeds [3].

VFS IDC: Quad Bi-Plane Breakdown

- **Stability (4)**: This configuration achieves high stability as a result of its structural rigidity and high moment of inertia in the hover phase. The efficiency results from the increase in the inertial resistance in the airframe allows for better passive gust rejection [7].
- **Hover Power (3):** The configuration obtains low disk loading as an advantage of the dedicated lift rotors, but it also has some extra weight from the second wing that offsets the power obtained [2], [7].
- **Endurance (2):** The configuration suffers from high interference drag between the wings and parasitic drag from the support struts. This overall creates a penalty in which more battery drain occurs [7].
- **Stall Speed (4):** The biplane arrangement of the configuration allows for a higher lift coefficient meaning that the speed for stall to occur is among the least out of the group. This allows for ultra slow speeds to optimize sensor performance [7], [9].
- **Mechanical Complexity (2):** The design and manufacturing of this drone type is very difficult due to the needed precision between the geometry of the wings, struts, and tension wires. This would increase cost, time to produce, and adds extra risks of failure to its mission [7].
- **Payload Delivery Accuracy (2):** While structurally stable, the dual-wing configuration doubles the surface area affected by the wind, emphasizing the “Sail Effect” when trying to drop the payload. This would require the drone to aggressively move to offset the wind which makes payload delivery difficult [2], [7], [9].
- **Cruise Speed (1):** The magnitude of the parasitic drag when compared to the other configurations is the highest during cruise, making the overall cruise speed the slowest [2], [7].

VFS IDC: Tiltwing Breakdown

- **Stability (1)**: This configuration detrimentally suffers to crosswinds, especially during its hover and transitioning phase, as a result of the wings acting in the "Sail Effect". This causes instability when approached by heavy winds as the surface area proportional to the wind increases as a result of the vertical wings in hover [5].
- **Hover Power (1):** The rear rotors ingest the turbulence created by the front rotors which causes thrust losses up to 25% on the rear. Already in high altitudes, this causes a detrimental loss in power increasing the chances of stall [8].
- **Endurance (3):** As a result of the inefficiencies in thrust power, the configuration often has to use increased amounts of power, but the aerodynamic body allows the aircraft to cruise easier without using as much power. This overall balances out power usage [3], [8].
- **Stall Speed (2):** Similar to the tiltrotor, this aircraft highly suffers from the possibility of stall during its transition phase. If the wings tilt too slow relative to the airspeed, the entire structure stalls easily [4].
- **Mechanical Complexity (1):** This design receives the lowest score because the wings are added to the tilting mechanism requiring even more torque and power consumption. This also exponentially increases the risk of failure from more intricate components needed to be utilized in its manufacturing.
- **Payload Delivery Accuracy (1):** Due to the nature of the vertically tilted wings during hover, the "Barn Door Effect" presents itself making the drone suffer in stability. Overall, this makes an inability to properly hover making the mission nearly impossible with this configuration [5].
- **Cruise Speed (3):** The design achieves high cruising speed by aligning the rotors and wings with the flow of its travel. This increases the speed of the drone, but also results in more mechanical complexity to its design [3].

VFS IDC: Final Four Design Matrix

Parameter	Weight	Lift + Cruise ✓	Tiltrotors ✗	Quad Bi-Plane ✗	Tiltwing ✗
Stability (Control Authority and Dynamic Response)	30	3	2	4	1
Hover Power	25	4	2	3	1
Endurance (Flight Time)	10	4	1	2	3
Stall Speed	10	3	1	4	2
Mechanical Complexity / Manufacturability	10	4	3	2	1
Payload Delivery Accuracy	10	4	3	2	1
Cruise Speed	5	2	4	1	3
Total	100	350	210	300	140

Detection Sensor Suite

Detection Sensor Requirements

- Mission success data requirements:
  - The aircraft needs to the know its global position to navigate to the search area.
  - The aircraft needs to know its velocity and angular acceleration to stabilize itself during the harsh flight.
  - The aircraft needs to know its distance from the ground to accurately descend to 100 ft above ground level.
  - The aircraft needs to know the terrain below it to swiftly locate the lost hiker and deliver the payload.
  - The aircraft needs to be able to recognize any obstacles in its surroundings to prevent any collisions.

Sensor Trade Study

Criteria	Weight	IMU	Long Range LIDAR	RTK GNSS	Ultrasonic	mmWave FMCW Radar	UWB Radar	Infrared	SAR Radar	Altimeter
Power Use	30	4	3	3	4	2	2	4	1	2
Cost	5	4	2	3	4	4	4	1	2	2
Range	15	4	4	4	1	3	2	4	4	4
Weight	20	4	3	2	2	2	1	4	2	3
Effectiveness	40	3	2	4	1	3	2	4	4	4
Totals		364	273	331	214	259	191	386	273	318

Sensor Component 1

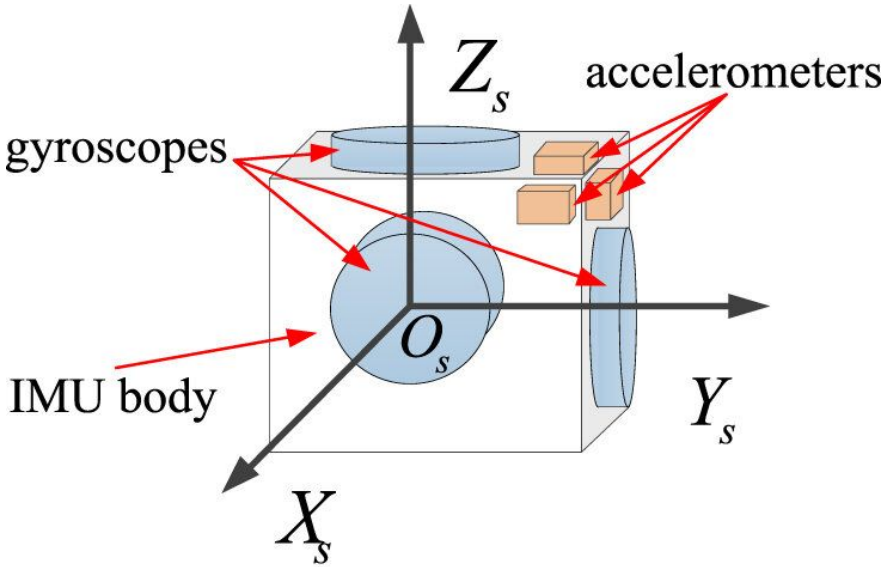
- After determining what information/data is needed, for each data type identify what component type your aircraft will use to obtain it, and create a slide with:
  - What the component is called
  - Why it's suited for your mission
  - Alternative solutions, and why the component you choose is best for the mission
  - What qualities you would look for if you were looking for a specific one to buy
- It's not required to identify a specific component brand, only identify the TYPE
  - For example, you don't have to say: "We're selecting the Hawkridge E38 GPS Mark II because of XYZ"
  - You can just say: "We're selecting a GPS because of XYZ" , "Important qualities of a GPS for this mission would be high refresh rate due to high flight speeds, and light weight due to weight constraints"
    - Obviously neither of the above are true for this mission!
- Each data type doesn't have to have its own component, sometimes components will take care of multiple data types!

Sensor Component 1

- RTK GNSS location sensor
  - This type of sensor is suitable for the mission as it allows the eVTOL to gather its own global location information to know where it is in relation to the search site and its starting location. Important qualities of an RTK GNSS sensor include increased accuracy as it takes signal from different GNSS systems simultaneously and uses a base station that receives the same signals but remains stationary to calculate its location with a variation of ~1cm. The reason why this is required is that satellite signals can be skewed by clouds or other disruptions so an RTK ensures that it will always get an accurate position.
  - An alternate solution for an RTK for location would've been a GPS, which can still work with clouded weather but cannot ensure precise results as the RTK that can lead to a delay in the delivery of the package which is unwanted in the given scenario.
  - If we were looking for a product we would try to attain a lightweight model that doesn't consume over 2-4 W/s of power to be sure to have energy efficient for the mission time constraint.

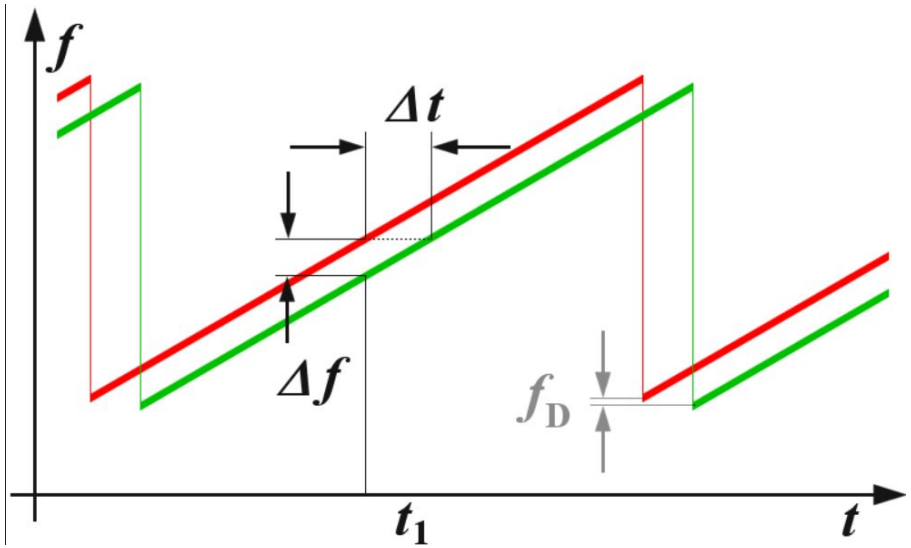
Sensor Component 2

- IMU (Inertial Measurement Unit)
  - This type of sensor is meant to be used for gathering information for the flight control system to stabilize the aircraft during flight. This will allow it to traverse the air efficiently despite gusts of wind. The sensor will read angular velocity and rotation to derive velocity and flight direction
  - Another alternative for an IMU sensor when it comes to flight stabilization is a optoelectronic motion capture system or OMC, which uses cameras and markers to track complex movement. However, the reason why the IMU is more efficient for eVTOL stabilization is that OMC is only used in controlled environment and takes measurements based on visual tools, both of which are lacking in the given scenario.
  - For purchasing an IMU, some specific characteristics we would focus on energy efficiency with it consuming less than 1 W/s and a model that is lightweight. Then for the components we would find one with quality accelerometers and gyroscopes, as these take in the main data for the IMU, and that included a barometer and magnetometer as these can help with stability.



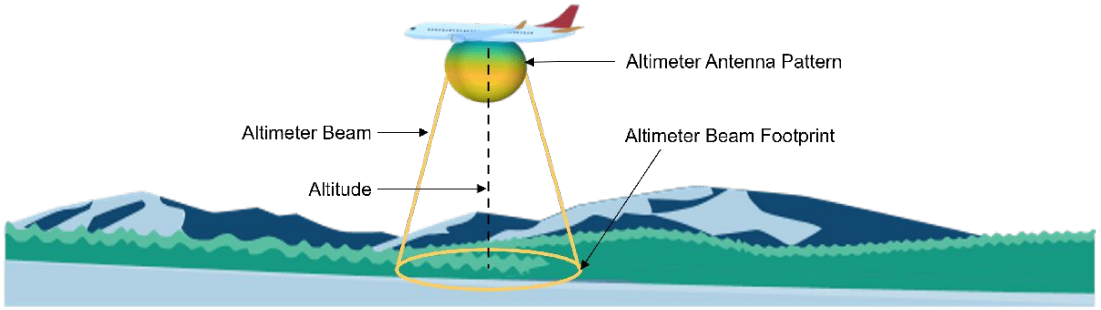
Sensor Component 3

- mmWave FMCW Radar
  - This type of sensor is suitable for the mission as it allows the eVTOL to avoid obstacles in its flight path with little trouble as the radio waves this type of radar uses is not interrupted by cloud cover. These sensors send out a “chirp” which is a high frequency radio wave that increases its frequency linearly over time.
  - An alternative for obstacle detection would be a LIDAR sensor as these are widely used for other kinds of drones and eVTOL. The issue with LIDAR in this scenario is that the water droplet in the clouds disrupt the signals it receives as its laser pulses never reach the desired surfaces, instead the pulses are either absorbed, reflected, or scattered by the dense water droplets. Because the radio waves used by the radar barely get disrupted by the droplets, allowing them to pierce through clouds, the radar is a much more suitable sensor for this mission.
  - For purchasing the mm Wave FMCW Radar we would pivot towards a model that consumes least energy as possible considering this is a powerful tool, has low weight version, and that has a range that is sufficient to keep the eVTOL from needing any sudden movements to avoid obstacles.



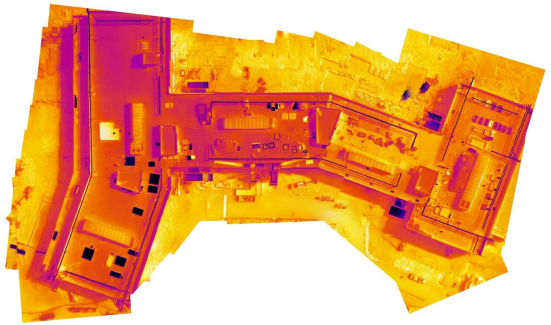
Sensor Component 4

- Radar Altimeter
  - This type of sensor is suitable for the mission as it allows the eVTOL to determine its height from the ground, allowing it to effectively descend to 100ft when dropping off the payload.
  - Some alternatives to the radar altimeter include the barometric altimeter, gps, and optical altimeter, which each have their own issues, but generally the cloud coverage keeps most of them from functioning properly, an issue the radar does not possess. From the options, the barometric altimeter is the only one unaffected by clouds, but it fails to accurately get the distance from the ground as it measures pressure, which can be distorted by weather and isn't consistent with terrain shifts. Leaving radar altimeters to be the only viable option as it is even capable of measuring the distance to up to 3000ft.
  - For purchasing a radar altimeter we only require one that has a range higher than the maximum distance from the ground that can be achieved on Mount Rainier which is around 5000 ft in the air.



Sensor Component 5

- Dual Radiometric Thermal & Visible Module
  - This sensor uses Long-Wave Infrared (LWIR) technology (8–14 µm), which is physically capable of penetrating the mission's "1/2 visibility fog" and canopy gaps to detect the hiker's body heat (37°C) against the cold mountain background. Its detection range allows for effective scanning at the safe flight altitude provided while scanning over the mountain range.
  - For the detection of the hiker we considered several different methods including visual detection and SAR radar sensor. The issue with a visual solution would be the fog and clouds as they would effectively blind the visual detection which was already inefficient. On the other hand, while the SAW radar sensor could detect shifts in the terrain, it would present a large constraint on both weight and power, both resources that are critical for completing this scenario effectively.
  - When sourcing this component, priority specifications include Resolution (minimum 640x512 px for human recognition at altitude), Frame Rate (60 Hz is required to prevent image blurring at search speeds of 25+ knots), SWaP (Size, Weight, and Power must be <100g and <5W), and a Radiometric Core to distinguish human body temperature from inanimate "false positives" like heated rocks.



Hiker Search Algorithm

Preliminary DSS Values

- Report the preliminary values of $\bar{y}, \theta, \bar{z}, v_s, A$ that you determined for the variables shown in the guide.
- Provide justification for all of the values, with any supporting calculations and factors of safety (FOS).
  - You can show screenshots of Python Code/Output, highlighting important parts
- Split this into multiple slides as necessary

Preliminary DSS Values

Based on the previous analysis of the Teledyne FLIR Hadron 640R (specifically the Boson+ 640 radiometric thermal core with a **32° HFOV / 13.6mm lens**):

- z = **60** meters
- theta = 32 degrees
- y = 34.4 meters
- v = 15 m/s
- A = 516 m^2/s

Preliminary DSS Values

Altitude (z = 60)

- **Justification:** We calculated the **Ground Sample Distance (GSD)** at 60m to be **5.4 cm/pixel**.
  - A human shoulder width is ~50cm.
  - Target size in pixels = $50 / 5.4 \approx 9.2 \text{ pixels}$.
  - **Johnson's Criteria** requires ~6 pixels for *recognition*.
- **Factor of Safety:** This provides a **1.5x FOS** over the minimum pixel requirement, allowing for detection even if the hiker is partially obscured by canopy or angled away.

2. Velocity (v = 15)

- **Justification:** The Hadron 640R operates at **60 Hz**.
  - At 15 m/s, the drone moves **0.25 meters per frame**.
  - The sensor sees 34.4 meters of terrain vertically (assuming square aspect for safety).
- **Factor of Safety:** There is **>99% overlap** between consecutive frames, ensuring that a single point on the ground is imaged ~130 times as the drone passes over. This eliminates "gaps" caused by signal noise.

3. Field of View (theta = 32)

- **Justification:** This is the native specification of the 13.6mm lens.
- **Note:** We assume the narrower HFOV (32°) rather than the visible camera FOV (67°) to ensure the *limiting factor* (the thermal sensor) dictates the flight path.

Search Algorithm Methodology

To ensure that the hiker was located in a strict 30-minute window, a python search algorithm was developed. Three distinct shapes were organized to determine the best possible pattern: Parallel Track (Lawnmower), Outward Spiral, and Inward Spiral. The simulation modeled a search radius of 1 mile or an area of approximately 3.14 square miles. Optimized for our sensor scanning width of 755 meters and a ground speed of 9.94 meters per second we obtained the figures below.

Search Algorithm Methodology

```python
# --------- 1. LAWNMOWER SEARCH PATTERN ---------
def lawnmower_search_time(searchDiameter, searchArea, scanWidth, velocity, nTrials):
    """
    Lawnmower pattern optimized for circular search area.

    Pattern: Parallel strips running across the circular area. Each strip is a
    straight line (chord) that intersects the circle. Strips near the center are
    longer (approaching full diameter), while strips near the edges are shorter.
    The aircraft flies each chord, then makes a perpendicular turn to the next
    parallel chord, spaced by the scanning width apart.

    Shape covered: Series of parallel rectangular strips that collectively
    approximate the circular search area. Some strip area extends beyond the
    circle at the edges, creating slight overlap waste.
    """
    radius = searchDiameter / 2.0

    # Number of strips required (round up for full coverage)
    nStrips = math.ceil(searchDiameter / scanWidth)

    # Calculate max distance and coverage
    maxDistance = 0
    coveredArea = 0
    stripInfo = []  # Store strip information for time-to-target calculation

    for i in range(nStrips):
        # Distance from center to strip centerline
        stripOffset = -radius + (i + 0.5) * scanWidth

        # Check if strip intersects circle
        if abs(stripOffset) < radius:
            # Chord length at this offset using: chord = 2*sqrt(r^2 - d^2)
            chordLength = 2 * math.sqrt(radius**2 - stripOffset**2)
            maxDistance += chordLength

            # Store strip info: (stripNumber, yPosition, chordLength, cumulativeDistance)
            stripInfo.append((i, stripOffset, chordLength, maxDistance))

            # Area covered by this strip (rectangular approximation)
            coveredArea += chordLength * scanWidth

        # Add turn distance if not the last strip
        if i < nStrips - 1:
            maxDistance += scanWidth

    # Max time to cover entire area
    maxTime = maxDistance / velocity

    # Calculate average time to reach random targets
    timeToTargets = []

    for trial in range(nTrials):
        targetX, targetY = generate_random_point_in_circle(radius)

        # Find which strip contains this target
        # Strips run horizontally (parallel to x-axis), so check y-coordinate
        distanceToTarget = 0
        found = False

        for i, (stripNum, yPos, chordLen, cumDist) in enumerate(stripInfo):
            # Check if target's y-coordinate falls within this strip's swath
            if abs(targetY - yPos) <= scanWidth / 2:
                # Target is in this strip
                # Distance = all previous strips + distance along this strip to target
                if i == 0:
                    distanceToTarget = cumDist - chordLen + abs(targetX + chordLen/2)
                else:
                    prevCumDist = stripInfo[i-1][3]
                    distanceToTarget = prevCumDist + scanWidth + abs(targetX + chordLen/2)
                found = True
                break

        if found:
            timeToTargets.append(distanceToTarget / velocity)

    avgTimeToTarget = sum(timeToTargets) / len(timeToTargets) if timeToTargets else maxTime

    # Coverage metrics
    overagePercent = ((coveredArea - searchArea) / searchArea) * 100

    print("\n" + "-"*60)
    print("LAWNMOWER PATTERN (Circular Optimized)")
    print("-"*60)
    print("Pattern shape: Parallel chords across circle")
    print("Number of strips needed: {}".format(nStrips))
    print("Total flight distance: {:.1f} m".format(maxDistance))
    print("Max search time (full area): {:.1f} s ({:.1f} min)".format(maxTime, maxTime/60))
    print("Avg time to random target: {:.1f} s ({:.1f} min)".format(avgTimeToTarget, avgTimeToTarget/60))
    print("Target area: {:.1f} m²".format(searchArea))
    print("Covered area: {:.1f} m²".format(coveredArea))
    print("Area overage: {:.2f}% (excess coverage beyond target)".format(overagePercent))

    return nStrips, maxDistance, maxTime, avgTimeToTarget
```

```python
# --------- 2. EXPANDING SPIRAL PATTERN ---------
def spiral_search_time(searchDiameter, searchArea, swathWidth, velocity, nTrials):
    """
    Expanding Archimedean spiral starting from center and spiraling outward.

    Pattern: Continuous spiral path starting at the center point and expanding
    outward in a smooth curve. Each complete revolution increases the radius by
    the scanning width, ensuring full coverage with no gaps. The spiral continues
    until reaching the outer edge of the circular search area.

    Shape covered: Smooth circular coverage that naturally matches the circular
    search area. Minimal wasted coverage since the spiral follows the natural
    geometry of the circle.
    """
    areaRadius = searchDiameter / 2.0

    # Each loop outward: radius increases by swathWidth
    nLoops = math.ceil(areaRadius / swathWidth)

    # Calculate total distance by summing circumferences
    maxDistance = 0
    coveredArea = 0

    for i in range(nLoops):
        radius_i = swathWidth * (i + 1)

        # Don't exceed the search area
        if radius_i > areaRadius:
            radius_i = areaRadius
            arcLength = 2 * math.pi * radius_i
            maxDistance += arcLength
            coveredArea = math.pi * areaRadius**2
            break
        else:
            arcLength = 2 * math.pi * radius_i
            maxDistance += arcLength
            coveredArea = math.pi * radius_i**2

    maxTime = maxDistance / velocity

    # Calculate average time to reach random targets
    timeToTargets = []

    for trial in range(nTrials):
        targetX, targetY = generate_random_point_in_circle(areaRadius)
        targetRadius = math.sqrt(targetX**2 + targetY**2)

        # Find which loop the target is in
        loopNum = math.ceil(targetRadius / swathWidth)
        if loopNum < 1:
            loopNum = 1

        # Distance = sum of all loops up to and including target loop
        distanceToTarget = 0
        for i in range(loopNum):
            r = swathWidth * (i + 1)
            if r > areaRadius:
                r = areaRadius
            distanceToTarget += 2 * math.pi * r

        # Approximate partial arc within final loop
        # Average case: target is halfway through the loop
        finalLoopRadius = min(swathWidth * loopNum, areaRadius)
        distanceToTarget -= math.pi * finalLoopRadius  # Subtract half loop (avg case)

        timeToTargets.append(distanceToTarget / velocity)

    avgTimeToTarget = sum(timeToTargets) / len(timeToTargets)

    # Coverage metrics
    overagePercent = ((coveredArea - searchArea) / searchArea) * 100

    print("\n" + "-"*60)
    print("OUTWARD SPIRAL PATTERN (From Center)")
    print("-"*60)
    print("Pattern shape: Archimedean spiral expanding outward")
    print("Number of spiral loops: {}".format(nLoops))
    print("Total flight distance: {:.1f} m".format(maxDistance))
    print("Max search time (full area): {:.1f} s ({:.1f} min)".format(maxTime, maxTime/60))
    print("Avg time to random target: {:.1f} s ({:.1f} min)".format(avgTimeToTarget, avgTimeToTarget/60))
    print("Target area: {:.1f} m²".format(searchArea))
    print("Covered area: {:.1f} m²".format(coveredArea))
    print("Area overage: {:.2f}% (excess coverage beyond target)".format(overagePercent))

    return nLoops, maxDistance, maxTime, avgTimeToTarget
```

```python
# --------- 3. INWARD SPIRAL PATTERN ---------
def inward_spiral_search_time(searchDiameter, searchArea, swathWidth, velocity, nTrials):
    """
    Contracting Archimedean spiral starting from perimeter and spiraling inward.

    Pattern: Continuous spiral path starting at the outer edge of the search area
    and spiraling inward toward the center. Each complete revolution decreases the
    radius by the scanning width. The spiral continues until reaching the center
    point of the circular search area.

    Shape covered: Smooth circular coverage identical to outward spiral but with
    reversed direction. Naturally matches the circular search area with minimal
    wasted coverage.
    """
    areaRadius = searchDiameter / 2.0

    # Each loop inward: radius decreases by swathWidth
    nLoops = math.ceil(areaRadius / swathWidth)

    # Calculate total distance by summing circumferences from outside to inside
    maxDistance = 0
    coveredArea = searchArea  # Spiral covers the full circular area

    for i in range(nLoops):
        radius_i = areaRadius - (swathWidth * i)

        if radius_i <= 0:
            break

        arcLength = 2 * math.pi * radius_i
        maxDistance += arcLength

    maxTime = maxDistance / velocity

    # Calculate average time to reach random targets
    timeToTargets = []

    for trial in range(nTrials):
        targetX, targetY = generate_random_point_in_circle(areaRadius)
        targetRadius = math.sqrt(targetX**2 + targetY**2)

        # For inward spiral, we reach outer targets first
        # Distance = sum of loops from outside until we reach target radius
        distanceToTarget = 0
        currentRadius = areaRadius

        for i in range(nLoops):
            r = areaRadius - (swathWidth * i)
            if r <= 0:
                break

            if r >= targetRadius:
                distanceToTarget += 2 * math.pi * r
                currentRadius = r
            else:
                # Target is between this loop and previous
                # Add partial arc (average: half loop)
                distanceToTarget += math.pi * currentRadius
                break

        timeToTargets.append(distanceToTarget / velocity)

    avgTimeToTarget = sum(timeToTargets) / len(timeToTargets)

    # Coverage metrics
    overagePercent = ((coveredArea - searchArea) / searchArea) * 100

    print("\n" + "-"*60)
    print("INWARD SPIRAL PATTERN (From Perimeter)")
    print("-"*60)
    print("Pattern shape: Archimedean spiral contracting inward")
    print("Number of spiral loops: {}".format(nLoops)
    print("Total flight distance: {:.1f} m".format(maxDistance))
    print("Max search time (full area): {:.1f} s ({:.1f} min)".format(maxTime, maxTime/60))
    print("Avg time to random target: {:.1f} s ({:.1f} min)".format(avgTimeToTarget, avgTimeToTarget/60))
    print("Target area: {:.1f} m²".format(searchArea))
    print("Covered area: {:.1f} m²".format(coveredArea))
    print("Area overage: {:.2f}% (excess coverage beyond target)".format(overagePercent))

    return nLoops, maxDistance, maxTime, avgTimeToTarget

# -- Run all three patterns --
lm_strips, lm_dist, lm_maxtime, lm_avgtime = lawnmower_search_time(searchDiameter, searchArea, scanningWidth, velocity, nTrials)
sp_loops, sp_dist, sp_maxtime, sp_avgtime = spiral_search_time(searchDiameter, searchArea, scanningWidth, velocity, nTrials)
in_loops, in_dist, in_maxtime, in_avgtime = inward_spiral_search_time(searchDiameter, searchArea, scanningWidth, velocity, nTrials)

# -- Summary comparison --
print("\n" + "-"*60)
print("COMPARISON SUMMARY")
print("-"*60)
print("Pattern          | Distance (m) | Max Time (min) | Avg Time (min)")
print("-"*60)
print("lawnmower        | {:>11.1f}  | {:>13.1f}  | {:>13.1f}".format(lm_dist, lm_maxtime/60, lm_avgtime/60)
print("Outward Spiral   | {:>11.1f}  | {:>13.1f}  | {:>13.1f}".format(sp_dist, sp_maxtime/60, sp_avgtime/60))
print("Inward Spiral    | {:>11.1f}  | {:>13.1f}  | {:>13.1f}".format(in_dist, in_maxtime/60, in_avgtime/60))
print("-"*60)
print("\nAverage Time Comparison (vs Lawnmower):")
print("Outward Spiral: {:+.1f}%".format((sp_avgtime-lm_avgtime)/lm_avgtime*100))
print("Inward Spiral:  {:+.1f}%".format((in_avgtime-lm_avgtime)/lm_avgtime*100))
```

First Iteration Search Flight Results

```
# -- CONSTANTS --
searchDiameter = 1*1609.344*2                    # [m] 1 mile radius area diameter
searchArea = math.pi*(searchDiameter/2)**2       # [m^2] total search area (circular)
scanningWidth = 840*math.sin(32*2*math.pi/180)  # [m] swath width at 840m AGL & 32° FOV
velocity = 8.94                                 # [m/s] aircraft ground speed; limited by the infared sensor
nTrials = 2000                                   # Number of random points to test for average time
```

```
============================================================
COMPARISON SUMMARY
============================================================
Pattern          | Distance (m) | Max Time (min) | Avg Time (min)
------------------------------------------------------------
Lawnmower        |     13803.5  |          25.7  |          20.4
Outward Spiral   |     24343.0  |          45.4  |          17.7
Inward Spiral    |     16104.3  |          30.0  |          28.3
============================================================
```

The values are not feasible as it only considers 1 trial which includes a single randomized position for the hiker which does not account for variability.

Final Iteration Search Flight Results

```
# -- CONSTANTS --
searchDiameter = 1*1609.344*2                    # [m] 1 mile radius area diameter
searchArea = math.pi*(searchDiameter/2)**2       # [m^2] total search area (circular)
scanningWidth = 840*math.sin(32*2*math.pi/180)  # [m] swath width at 840m AGL & 32° FOV
velocity = 8.94                                 # [m/s] aircraft ground speed; limited by the infared sensor
nTrials = 2000                                   # Number of random points to test for average time
```

```
================================================================
COMPARISON SUMMARY
================================================================
Pattern          | Distance (m) | Max Time (min) | Avg Time (min)
----------------------------------------------------------------
Lawnmower        |      13803.5 |           25.7 |           12.3
Outward Spiral   |      24343.0 |           45.4 |           17.3
Inward Spiral    |      16104.3 |           30.0 |           29.9
================================================================
```

Aircraft Sizing

Wing Sizing - Constraints, pt. 1 (V_{stall}, Blades, Energy)

(1) $V_{stall}(FoS) < V_{cruise}$

where $V_{stall} = \sqrt{\frac{MTOW * hover_{fraction}}{qSC_{L,max}}}$, FoS = 1.1, $V_{cruise} = 8.94\ m/s$,

in which V_{cruise} is a constraint derived from frame of the thermal sensor.

```
def stall_speed(W_cruise, S_cruise,hover_frac_search):
    """
    Returns stall speed [m/s] for current config.
    """
    W_felt = W_cruise*(1-hover_frac_search)
    V_stall_cruise = math.sqrt((2 * W_felt) / (rho_cruise * S_cruise * C_L_max_cruise))
    # print(V_stall_cruise)
    return V_stall_cruise
```

(2) $D_{rotor,combinated} + x_{props} < S_{wing} - S_{fuselage}$

where $D_{rotor,combinated} = 2 * \sqrt{\frac{A_{hover}}{4\pi}}$, non-rotor distances on the wingspan are

$x_{props} + S_{fuselage} \approx 1.1 * D_{rotor,combinated}$,

in which A and S are simulation-iterated values

(3) $E_{usable} / E_{mission} \geq 100\%$

where E_{usable} is first calculated,

Wing Sizing - Constraints, pt. 2 (Power, Energy)

```python
def battery_sizing(W_cruise, E_mission_Wh):
    """
    Calculates battery mass/usable Wh/energy ratio.
    """
    specific_energy_batt = 65 * 2.205            # Wh/kg
    battery_mass_batt = 0.3 * ((W_cruise - 22.24 - 0.381*9.81)/9.81)    # only place where mass of payload and sensors affect calculations
    # battery_mass_batt = 0.2 * (W_cruise/9.81)
    E_usable_batt = battery_mass_batt * specific_energy_batt * 0.8      # 0.8 for usable part of battery
    ratio_batt = E_usable_batt / E_mission_Wh * 100
    return battery_mass_batt, E_usable_batt, ratio_batt
```

and then is compared with the energy usage in the mission,

```python
def power_and_energy(W_cruise, S_cruise, A_hover, C_D_cruise,hover_frac_search):
    """
    Computes drag/thrust, induced velocities, segment powers, mission energy.
    Returns drag, induced (hover), P_cruise, P_hover, ascent/descent, energy in Wh.
    """
    D_cruise = q_cruise * S_cruise * C_D_cruise                    # Drag force, N
    v_induced_hover = math.sqrt(W_cruise / (2 * rho_cruise * A_hover)) # Induced hover velocity, m/s

    P_cruise = D_cruise * v_rel_avg_cruise / η_cruise
    P_hover = W_cruise * v_induced_hover / η_cruise
    P_ascent = 1.2 * P_hover
    P_descent = 0.9 * P_hover

    P_sensors_transport = 11.6
    P_sensors_search = 13.6

    t_cruise_travel = 20 * 60            # seconds
    t_cruise_search = 20 * 60            # seconds
    t_cruise = t_cruise_travel + t_cruise_search
    t_hover = 3 * 60
    t_ascent = 3 * 60
    t_descent = 3 * 60

    E_mission = (P_cruise * t_cruise + P_hover * t_hover +
                 P_ascent * t_ascent + P_descent * t_descent +
                 P_sensors_transport * t_cruise_travel + P_sensors_search * t_cruise_search +
                 P_hover * hover_frac_search * t_cruise_search)
    E_mission_Wh = E_mission / 3600 #conver to Wh

    return D_cruise, v_induced_hover, P_cruise, P_hover, P_ascent, P_descent, E_mission_Wh
```

(4) $$C_{airfoil_min} < C_L < C_{airfoil_max}$$

where S1223 airfoil are imported as an array,

```python
#S1223 airfoil data
alpha_array_cruise = np.array([
-7.25, -7.00, -6.75, -6.50, -6.25, -6.00, -5.75, -5.50, -5.25, -5.00, -4.75, -4.50, -4.25,
-3.75, -3.50, -3.25, -3.00, -2.75, -2.50, -2.25, -2.00, -1.75, -1.50, -1.25, -1.00, -0.75,
-0.50, -0.25, 0.00, 0.25, 0.50, 1.00, 1.25, 1.50, 1.75, 2.00, 2.25, 2.50, 2.75, 3.00, 3.25,
3.50, 3.75, 4.25, 4.50, 4.75, 5.00, 5.25, 5.50, 5.75, 6.00, 6.25, 6.50, 6.75, 7.00, 7.25,
7.50, 7.75, 8.00, 8.25, 8.50, 8.75, 9.00, 9.25, 9.50, 9.75, 10.00, 10.25, 10.50, 10.75,
11.00, 11.25, 11.50, 11.75, 12.00, 12.25, 12.50, 12.75, 13.00, 13.25, 13.50, 13.75, 14.00,
14.25, 14.50, 14.75, 15.00, 15.25, 15.50, 15.75, 16.00, 16.25, 16.50, 16.75, 17.00
])

cl_array_cruise = np.array([
0.1102, 0.1307, 0.1557, 0.2154, 0.2372, 0.1507, 0.1624, 0.1830, 0.2136,
0.2652, 0.3237, 0.3386, 0.3503, 0.3379, 0.3471, 0.3608, 0.4407, 0.4581,
0.4860, 0.8394, 0.8855, 0.9290, 0.9657, 0.9988, 1.0374, 1.0715, 1.1069,
1.1406, 1.1850, 1.2132, 1.2409, 1.2962, 1.3244, 1.3524, 1.3804, 1.4089,
1.4373, 1.4655, 1.4939, 1.5218, 1.5500, 1.5799, 1.6082, 1.6051, 1.6794,
1.7050, 1.7301, 1.7560, 1.7849, 1.8082, 1.8316, 1.8547, 1.8777, 1.9003,
1.9234, 1.9495, 1.9706, 1.9920, 2.0132, 2.0331, 2.0533, 2.0760, 2.0941,
2.1121, 2.1296, 2.1463, 2.1645, 2.1826, 2.2002, 2.2165, 2.2310, 2.2486,
2.2650, 2.2779, 2.2927, 2.3063, 2.3169, 2.3293, 2.3370, 2.3455, 2.3509,
2.3523, 2.3534, 2.3524, 2.3470, 2.3388, 2.3327, 2.3281, 2.3096, 2.2926,
2.2771, 2.2516, 2.2347, 2.2066, 2.1825
])

cd_array_cruise = np.array([
0.09424, 0.08089, 0.06065, 0.06577, 0.06215, 0.08059, 0.07791, 0.07507, 0.07167,
0.06686, 0.06171, 0.05955, 0.05680, 0.05385, 0.05148, 0.04986, 0.03815, 0.03829,
0.03691, 0.01585, 0.01490, 0.01439, 0.01417, 0.01418, 0.01397, 0.01396, 0.01398,
0.01398, 0.01374, 0.01392, 0.01419, 0.01448, 0.01464, 0.01484, 0.01511, 0.01530,
0.01544, 0.01561, 0.01579, 0.01599, 0.01624, 0.01659, 0.01676, 0.01710, 0.01699,
0.01726, 0.01755, 0.01790, 0.01837, 0.01859, 0.01885, 0.01912, 0.01941, 0.01972,
0.02011, 0.02059, 0.02087, 0.02118, 0.02151, 0.02183, 0.02227, 0.02284, 0.02317,
0.02352, 0.02392, 0.02444, 0.02503, 0.02543, 0.02590, 0.02647, 0.02722, 0.02773,
0.02831, 0.02912, 0.02986, 0.03066, 0.03171, 0.03266, 0.03399, 0.03532, 0.03692,
0.03894, 0.04187, 0.04349, 0.04644, 0.04985, 0.05321, 0.05751, 0.06179, 0.06719,
0.07270, 0.08081, 0.08627, 0.09454, 0.10230
])
```

then C_L is calculated using lift formula with the assumption of SLUF,

```python
#C_L_cruise calculated using the weight "felt" by the aircraft to account
    #for lift from vertical props in slow-speed cruise
C_L_cruise = (W_cruise - W_cruise*hover_frac_search) / (q_search * S_cruise)

#check if C_L_cruise go out of bounds
if C_L_cruise < np.min(cl_array_cruise) or C_L_cruise > np.max(cl_array_cruise):
    return None, None, None

AOA_cruise = np.interp(C_L_cruise, cl_array_cruise, alpha_array_cruise)
```

These constraints were then implemented in a nested for loop, referencing aforementioned functions for values.

Wing Sizing - Simulating Configurations

```python
def evaluate_designs(MTOW_range, S_range, A_range,hover_frac_search_range):
    """
    Loops through all sizings, assembles dicts for feasible designs.
    """
    results = []
    for hover_frac_search in frange(*hover_frac_search_range):
        for W_cruise in frange(*MTOW_range):
            for S_cruise in frange(*S_range):
                for A_hover in frange(*A_range):

                    # Geometry
                    sigma_cruise, sigma_hover, b_cruise, D_rotors_hover = wing_and_rotor_geometry(W_cruise, S_cruise, A_hover)
                    if b_cruise <= 1.1 * D_rotors_hover:
                        continue

                    # Aerodynamics
                    C_L_cruise, AOA_cruise, C_D_cruise = aerodynamic_coeffs(W_cruise, S_cruise, b_cruise,hover_frac_search)
                    if C_L_cruise is None:
                        continue

                    # Power/Energy
                    D_cruise, _, _induced_hover, P_cruise, P_hover, P_ascent, P_descent, E_mission_Wh = power_and_energy(W_cruise, S_cruise, A_hover, C_D_cruise,hover_frac_search)

                    # Battery
                    battery_mass_batt, E_usable_batt, ratio_batt = battery_sizing(W_cruise, E_mission_Wh)

                    # Stall speed and conversions
                    V_stall_cruise = stall_speed(W_cruise, S_cruise,hover_frac_search)
                    W_lbs_cruise = W_cruise / 4.44822
                    if (FoS*V_stall_cruise>v_search):
                        continue

                    # Record if feasible
                    if ratio_batt >= 100:
                        results.append({
                            "W_N_cruise": round(W_cruise, 2),
                            "W_lbs_cruise": round(W_lbs_cruise,2),
                            "sigma_cruise": round(sigma_cruise,2),
                            "sigma_hover": round(sigma_hover,2),
                            "S_cruise": round(S_cruise,2),
                            "A_hover": round(A_hover,2),
                            "b_cruise": round(b_cruise,2),
                            "D_rotors_hover": round(D_rotors_hover,2),
                            "V_stall_cruise": round(V_stall_cruise,2),
                            "E_usable_batt": round(E_usable_batt,2),
                            "E_mission_Wh": round(E_mission_Wh,2),
                            "AOA_cruise": round(AOA_cruise,2),
                            "C_L_cruise": round(C_L_cruise,2),
                            "C_D_cruise": round(C_D_cruise,4),
                            "LD_cruise": round(C_L_cruise/C_D_cruise,2),
                            "D_cruise": round(D_cruise,2),
                            "ratio_batt": round(ratio_batt,2),
                            "hover_frac_search": round(hover_frac_search,2)
                        })
    return results
```

The following range determines the breadth of the configuations calculated and step size defines the fidelity of the simulation.

```python
##########################################################################
# 4. Vehicle Sizing Ranges

MTOW_range = (25, 500, 1)              # [W_cruise]: N
S_range = (1, 30.0, 0.1)               # [S_cruise]: m^2
A_range = (1, 30.0, 0.1)               # [A_hover]: m^2
hover_frac_search_range = (0,1,0.01)   # [hover_frac_search]: unitless
    # the percent of max RPM that the lift rotors spin at during CRUISE in SEARCH
```

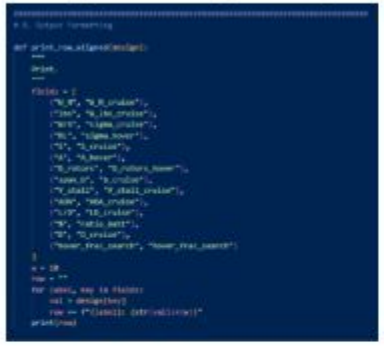
476 MTOW × 291 S × 291 A × 101 Hover Fraction results in 4,071,123,756 total simulation iterations

To sort these designs, a weighted scoring system is developed and defined in the following code,

Wing Sizing – Ranking Sims, Choosing Best

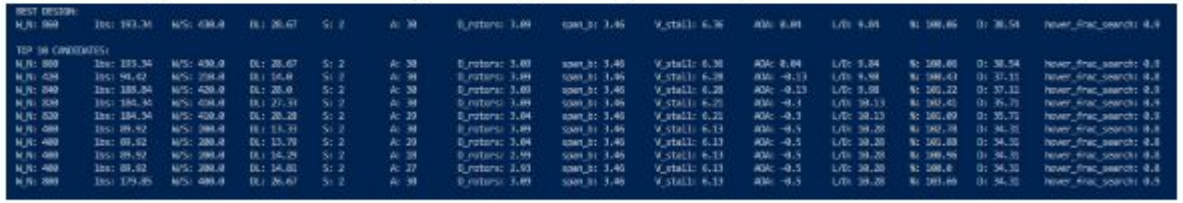
```python
##############################################################################
# 7. Scoring and Selection

def score_design(d):
    # Higher is better, penalize excess weight, stall speed, loading, drag.
    score = (
        + 10 * d["ratio_batt"]
        # - 2 * (d["W_N_cruise"] / 9.81)
        - 1000 * (d["V_stall_cruise"] - v_rel_avg_cruise)**2
        - 5 * d["sigma_cruise"]
        - 5 * d["sigma_hover"]
        - 0.2 * d["D_cruise"]
        + 0.5 * d["LD_cruise"]
        - 100*d["S_cruise"]
    )
    return score

def find_best_design(results, n=10):
    if not results:
        print("No feasible designs found!")
        return None, []
    ranked = sorted(results, key=lambda d: score_design(d), reverse=True)
    return ranked[0], ranked[:n]
```

Paired with the constraints aforementioned, the output will *only* return viable options, sorted by most feasible designs. For each of these configurations, relevant data including MTOW (in newtons and pounds), W/S (wing loading ratio), D/L (disk loading ratio), S (wing area), A (total rotor area), b (wingspan), stall velocity, angle of attack, L/D ratio, excess energy %, and hover_frac are outputted by the following code:



```python
##############################################################################
# 9. Main Algorithm

solutions = evaluate_designs(
    MTOW_range=MTOW_range,
    S_range=S_range,
    A_range=A_range,
    hover_frac_search_range=hover_frac_search_range
)
best, top10 = find_best_design(solutions)
print("BEST DESIGN:")
print_row_aligned(best)
print("\nTOP 10 CANDIDATES:")
for d in top10:
    print_row_aligned(d)
```

The final output with our best + top 10 design configuration specifications are as follows,



The conventional values for Oswald efficiency number, e=0.8, and propeller efficiency, η=0.7, are used. Calculation of these values would require advanced CFD and/or wind tunnel testing, which do not fit within the scope of this challenge.

Proof of Mission Success

Proof of Mission Success

```
BEST DESIGN:
W_N: 860    lbs: 193.34    W/S: 430.0    DL: 28.67    S: 2    A: 30    D_rotors: 3.09    span_b: 3.46    V_stall: 6.36    AOA: 0.04    L/D: 9.84    %: 100.06    D: 38.54    hover_frac_search: 0.9

TOP 10 CANDIDATES:
W_N: 860    lbs: 193.34    W/S: 430.0    DL: 28.67    S: 2    A: 30    D_rotors: 3.09    span_b: 3.46    V_stall: 6.36    AOA: 0.04    L/D: 9.84    %: 100.06    D: 38.54    hover_frac_search: 0.9
W_N: 420    lbs: 94.42    W/S: 210.0    DL: 14.0    S: 2    A: 30    D_rotors: 3.09    span_b: 3.46    V_stall: 6.28    AOA: -0.13    L/D: 9.98    %: 100.43    D: 37.11    hover_frac_search: 0.8
W_N: 840    lbs: 188.84    W/S: 420.0    DL: 28.0    S: 2    A: 30    D_rotors: 3.09    span_b: 3.46    V_stall: 6.28    AOA: -0.13    L/D: 9.98    %: 101.22    D: 37.11    hover_frac_search: 0.9
W_N: 820    lbs: 184.34    W/S: 410.0    DL: 27.33    S: 2    A: 30    D_rotors: 3.09    span_b: 3.46    V_stall: 6.21    AOA: -0.3    L/D: 10.13    %: 102.41    D: 35.71    hover_frac_search: 0.9
W_N: 820    lbs: 184.34    W/S: 410.0    DL: 28.28    S: 2    A: 29    D_rotors: 3.04    span_b: 3.46    V_stall: 6.21    AOA: -0.3    L/D: 10.13    %: 101.09    D: 35.71    hover_frac_search: 0.9
W_N: 400    lbs: 89.92    W/S: 200.0    DL: 13.33    S: 2    A: 30    D_rotors: 3.09    span_b: 3.46    V_stall: 6.13    AOA: -0.5    L/D: 10.28    %: 102.78    D: 34.31    hover_frac_search: 0.8
W_N: 400    lbs: 89.92    W/S: 200.0    DL: 13.79    S: 2    A: 29    D_rotors: 3.04    span_b: 3.46    V_stall: 6.13    AOA: -0.5    L/D: 10.28    %: 101.88    D: 34.31    hover_frac_search: 0.8
W_N: 400    lbs: 89.92    W/S: 200.0    DL: 14.29    S: 2    A: 28    D_rotors: 2.99    span_b: 3.46    V_stall: 6.13    AOA: -0.5    L/D: 10.28    %: 100.96    D: 34.31    hover_frac_search: 0.8
W_N: 400    lbs: 89.92    W/S: 200.0    DL: 14.81    S: 2    A: 27    D_rotors: 2.93    span_b: 3.46    V_stall: 6.13    AOA: -0.5    L/D: 10.28    %: 100.0    D: 34.31    hover_frac_search: 0.8
W_N: 800    lbs: 179.85    W/S: 400.0    DL: 26.67    S: 2    A: 30    D_rotors: 3.09    span_b: 3.46    V_stall: 6.13    AOA: -0.5    L/D: 10.28    %: 103.66    D: 34.31    hover_frac_search: 0.9
```

```
--- Best Design Summary ---
MTOW: 405 N (91.05 lbs)
Wing Area (S): 2 m²
Rotor Area (A): 30 m²
Wing Span (b): 3.46 m
Total Rotor Diameter: 3.09 m
Wing Loading: 202.5 N/m²
Disk Loading: 13.5 N/m²
Stall Speed: 6.17 m/s, Factor of Safety (FoS): 3.0430599675850893
Energy Margin (assumes 80% usage limit): 102.17%


--- Cruise Performance ---
Relative Velocity (v_rel_avg_cruise): 18.78 m/s
Lift Coefficient (C_L_cruise): 1.270
Angle of Attack (AOA_cruise): -0.40 deg
Drag Coefficient (C_D_cruise): 0.1098
Required Thrust (=D_cruise): 35.02 N
Induced Velocity (Cruise, v_induced_cruise): 0.803 m/s
```

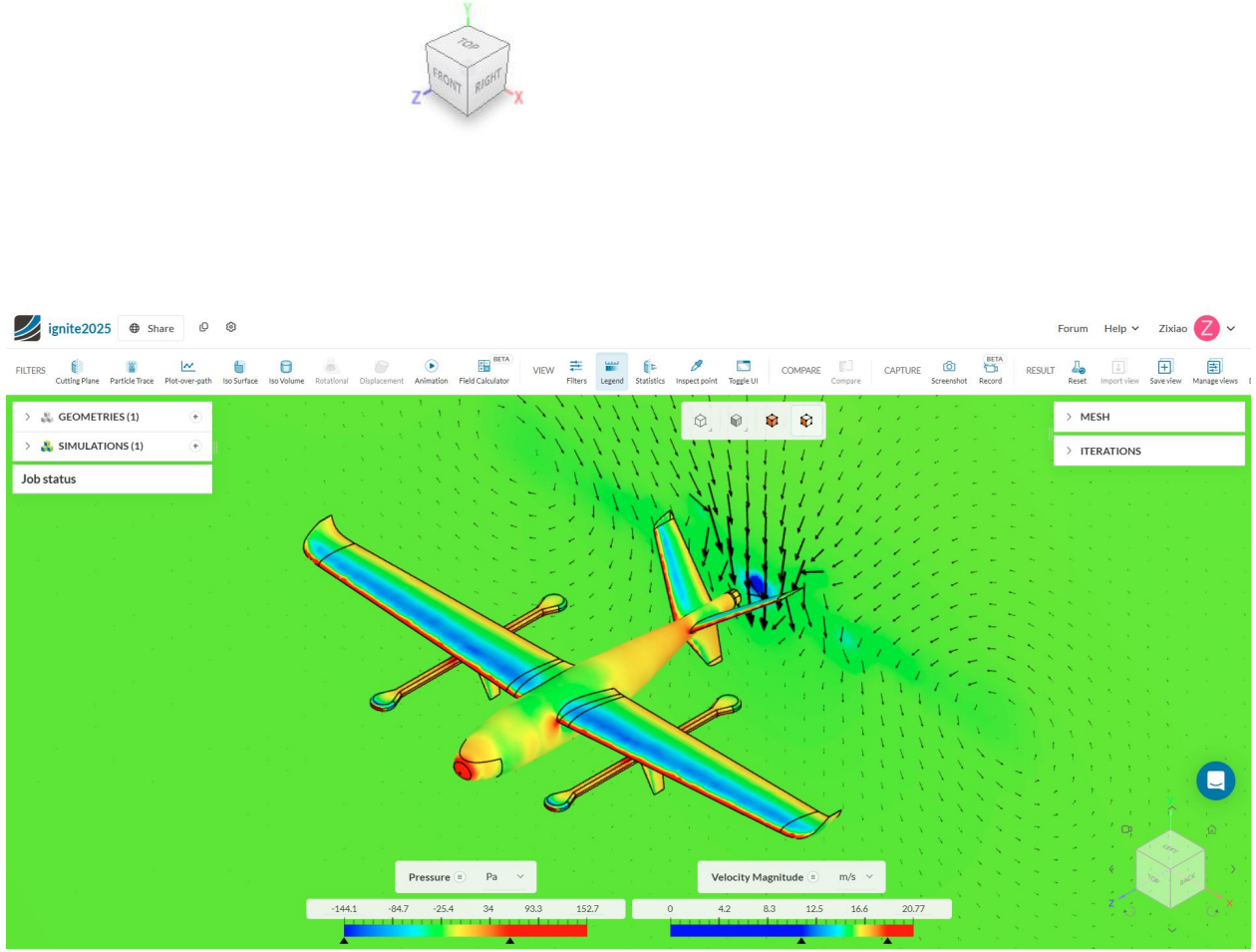
```
--- Propeller Information -- Cruise ---
Number Props (N_cruise): 1
Propeller Diameter (D_cruise): 1.335 m
Disk Area (A_cruise): 1.400 m²
Shaft RPM (rpm_cruise): 402.1
Torque (Q_cruise): 26.714 N·m
Pitch (pitch_cruise): 2.801 m/rev
Tip Speed: 28.12 m/s (Mach 0.086)

--- Propeller Information -- Hover ---
Number Props (N_hover): 4
Prop Diameter (D_hover): 3.090 m
Suggested RPM (rpm_limit_hover): 1420.7
Torque per Prop (Torque_per_hover): 3.320 N·m
Tip Speed Actual: 229.88 m/s (Mach 0.700)
Disk Loading (Per Prop): 13.50 N/m²
```

Proof of Mission Success

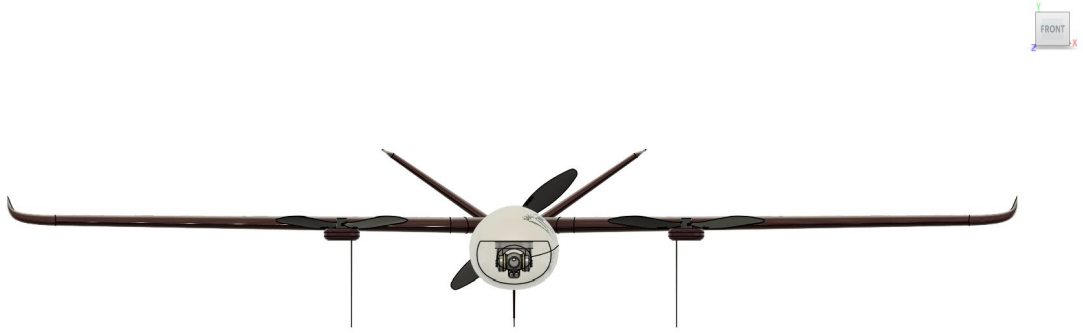
- The worst case scenario for the search segment would last 25.73 min whilst the worst travel time would be around 20 min, adding up to the worst case scenario taking around 45.73 minutes, which does not go over the allotted time for the delivery mission.
- Constraints (3) and (1) show that:
  - battery and MTOW combination will provide enough power to sustain the entire mission
  - aircraft will not stall at any point in the mission, respectively

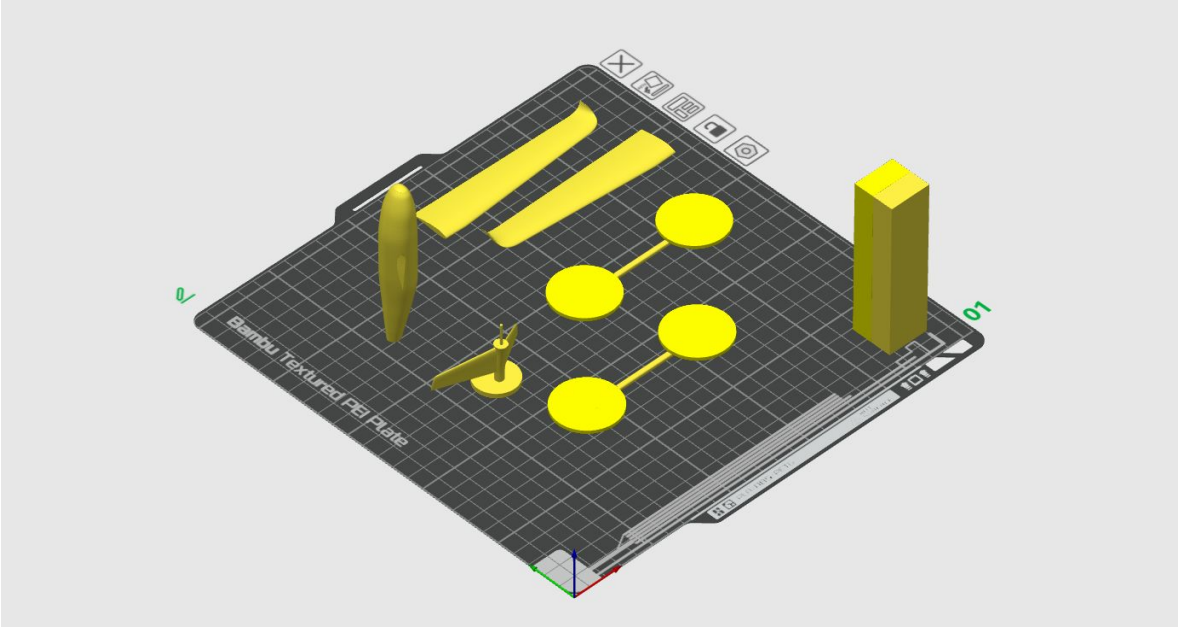
Extra Credit

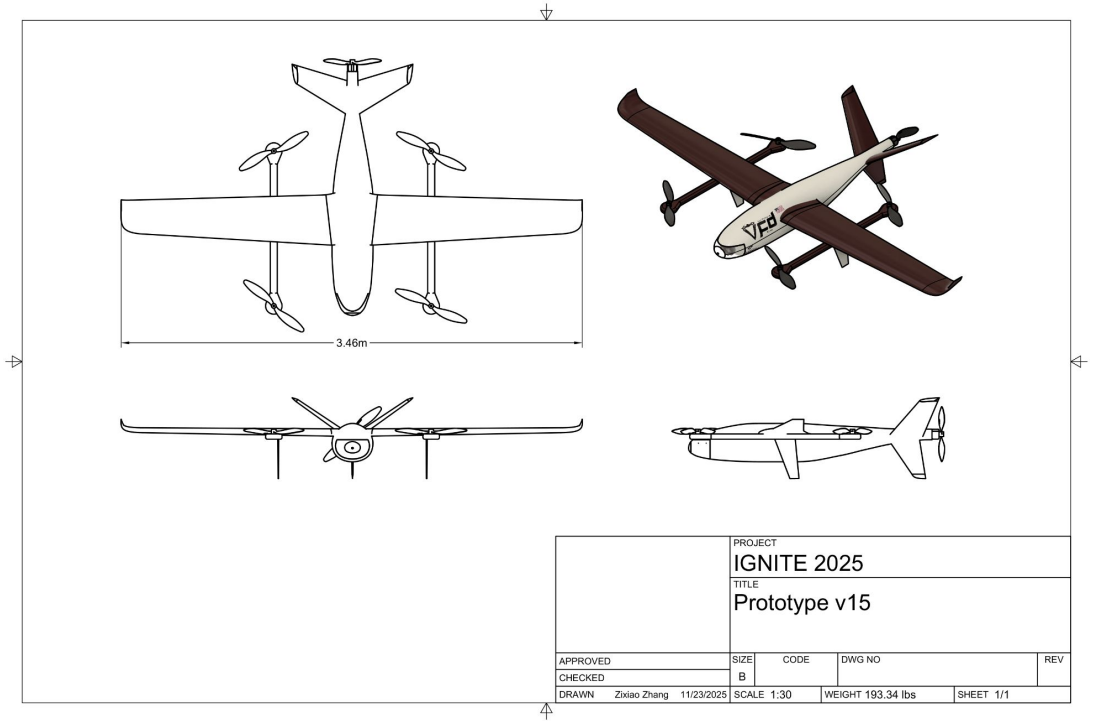
Extra Credit



Extra Credit







References & Additional Sources

References

- [1] J. Kaneshige, T. Lombaerts, K. Shish, and M. Feary, “Simplified Vehicle Control Concept for a Lift Plus Cruise eVTOL Vehicle,” AIAA AVIATION FORUM AND ASCEND 2024. American Institute of Aeronautics and Astronautics, July 27, 2024. doi: 10.2514/6.2024-4207.
- [2] E. Vargas Loureiro et al., “Evaluation of low fidelity and CFD methods for the aerodynamic performance of a small propeller,” Aerospace Science and Technology, vol. 108, p. 106402, Jan. 2021, doi: 10.1016/j.ast.2020.106402.
- [3] C. Johnson, A. M. Klem, and D. P. Trawick, “Range and endurance tradeoffs on personal rotorcraft design,” NASA, Tech. Rep. NASA/TM-2016-219159, 2017.
- [4] Z. Liang, L. Fan, G. Wen, and Z. Xu, “Design, Modeling, and Control of a Composite Tilt-Rotor Unmanned Aerial Vehicle,” Drones, vol. 8, no. 3, p. 102, Mar. 2024, doi: 10.3390/drones8030102.
- [5] M. Moore, “NASA Puffin Electric Tailsitter VTOL Concept,” 10th AIAA Aviation Technology, Integration, and Operations (ATIO) Conference. American Institute of Aeronautics and Astronautics, June 25, 2010. doi: 10.2514/6.2010-9345.

References

- [6] S. Xiang et al., “Autonomous eVTOL: A summary of researches and challenges,” Green Energy and Intelligent Transportation, vol. 3, no. 1, p. 100140, Feb. 2024, doi: 10.1016/j.geits.2023.100140.

- [7] F. Li, W.-P. Song, B.-F. Song, and J. Jiao, “Dynamic Simulation and Conceptual Layout Study on a Quad-Plane in VTOL Mode in Wind Disturbance Environment,” International Journal of Aerospace Engineering, vol. 2022, pp. 1–24, Jan. 2022, doi: 10.1155/2022/5867825.

- [8] A. Zanotti, A. Velo, C. Pepe, A. Savino, D. Grassi, and L. Riccobene, “Aerodynamic interaction between tandem propellers in eVTOL transition flight configurations,” Aerospace Science and Technology, vol. 147, p. 109017, Apr. 2024, doi: 10.1016/j.ast.2024.109017.

- [9] L. Qing, L. Juanxia, Z. Xingyu, and L. Juhong, “Research of Transition Corridor and Optimal Trajectory for Front-Tilt-Quadplane eVTOL Aircraft,” 2023 6th International Conference on Electronics Technology (ICET). IEEE, pp. 898–902, May 12, 2023. doi: 10.1109/icet58434.2023.10211673.